INF384 · TEMAS AVANZADOS EN INGENIERÍA DE SOFTWARE 1 · 2026-2

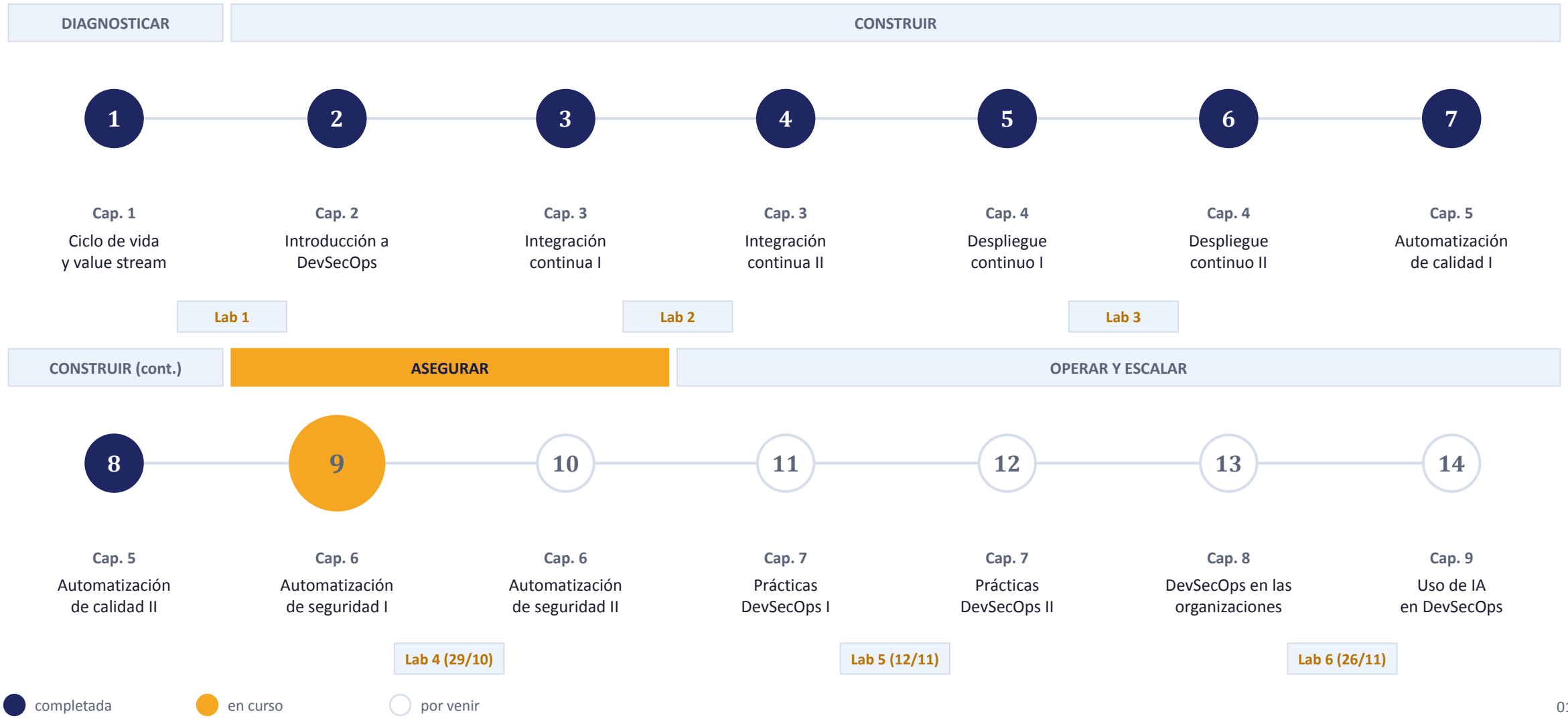
Sesión 09

Automatización de seguridad: código, dependencias y contenedores

Capítulo 6 (1 de 2) · 3 horas | RA3 | Prof. José Felipe del Castillo Salazar

INF384 · 9 CAPÍTULOS · 14 SESIONES · 42 HORAS

La ruta del curso



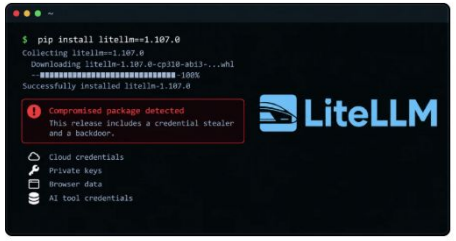
SESIÓN 9

Objetivo de la sesión y ruta

1. Análisis estático de seguridad del código
2. Análisis de librerías y componentes
3. Análisis de contenedores
4. La secuencia de controles sobre el artefacto

A. Actividad: triage de hallazgos por caso

Resultado de aprendizaje vinculado: RA3 — seguridad integrada en las etapas del ciclo de vida con análisis estático, de dependencias y de contenedores.



LiteLLM Compromised in Supply Chain Attack Delivering Credential Stealer and Backdoor

Oct 21, 2026 Malware / Artificial Intelligence

The popular LiteLLM Python package has been compromised as part of a software supply chain attack that delivered a credential stealer and a...



PHP Issues Emergency Patches for Two Critical PHP2Shell RCE Flaws as Exploit Attempts Emerge

Oct 21, 2026 Vulnerability / Web Security

PHP has released emergency updates that fix two critical flaws in the language itself, letting an attacker without an account run code on a web...



China-Aligned Bronze Sandcastle Abuses Sandboxie DLL Sideloading to Deploy Backdoors

Oct 21, 2026 Cyber Espionage / Malware

Cybersecurity researchers have disclosed details of a previously undocumented China-aligned threat cluster known as Bronze Sandcastle...



Attackers Hijacked Over 300 Tor Exit Relays to Intercept SSH Connections

Oct 21, 2026 Network Security / Cyber Espionage

Attackers took control of more than 300 Tor exit relays in early October and used them to redirect SSH connections to servers that stole login...



Attackers Abuse UpdraftCentral Keys to Plant Backdoors on WordPress Sites

Oct 21, 2026 Cyber Attack / Web Security

Unknown threat actors breached TeamUpdraft's website and potentially gained unauthorized access to customers' connection keys for...

Microsoft pins ConnectWise supply-chain attack to China-state aligned threat group

Storm-0940 used the agentic AI penetration testing framework to automate attacks and "reach more than 2,250 tenants at 2,900 sign-in attempts per minute," Microsoft researchers said.

BY MATT KAPKO • OCTOBER 21, 2026

Listen to this article 4:08 Learn more.





CONTINUIDAD CON LA SESIÓN 4

Dónde estamos en el pipeline

Commit	Build	Pruebas	Calidad	**Seguridad**	Publicar	Desplegar	Observar

Lo que la sesión 4 dejó resuelto

- Un control entra a integración continua si es rápido, determinista, acotable al cambio y no necesita ambiente.
- La política del gate detiene lo nuevo e inventaría lo heredado; la condición se escribe antes del hallazgo.
- La excepción tiene cuatro datos: identificador, razón, responsable y vencimiento.
- Una credencial en el repositorio se rota; nunca se exceptúa.

Lo que todavía no está decidido

- ¿Cómo decide un analizador que una línea de código es explotable?
- ¿Por qué un hallazgo de severidad crítica puede esperar y uno medio detener el pipeline?
- ¿Qué viaja dentro de la imagen además del código que escribió el equipo?
- ¿Qué hace el pipeline con un hallazgo que la herramienta marcó y nadie confirmó?

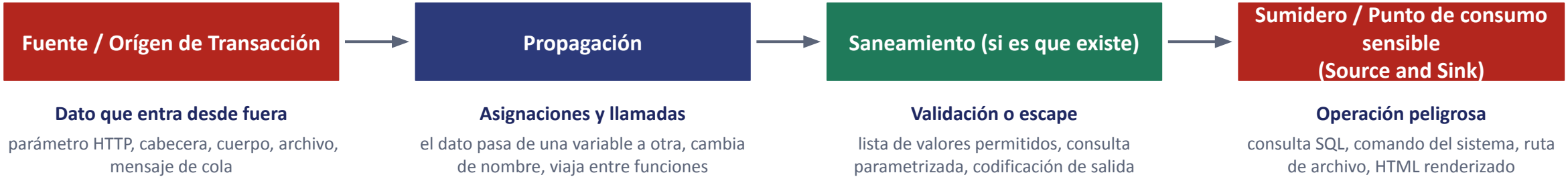
Hoy la etapa de seguridad se abre por dentro: cada control se trata como una pregunta sobre el artefacto, con su alcance y sus puntos ciegos.

A

Análisis estático de seguridad

Cómo encuentra el analizador un patrón explotable sin ejecutar el código, y qué se le escapa

Fuente, sumidero y el flujo entre ambos



1

Validación de Patrón: la línea coincide con una regla (segundos)

Una llamada a eval, una contraseña por defecto, un algoritmo de cifrado retirado. Rápido y con muchos avisos que no llevan a nada.

2

Validación de Flujo de datos: existe un camino de la fuente al sumidero sin pasar por saneamiento (minutos)

El analizador sigue el dato a través de funciones y archivos. Más lento, y el hallazgo trae el camino completo.

El hallazgo útil es el camino: dónde entró el dato, por dónde pasó y dónde se usó. Con eso el autor corrige en el punto exacto en lugar de buscar.

El mismo camino en el código

SRC/PEDIDOS.JS — VERSIÓN VULNERABLE Y CORRECCIÓN

```
export async function obtenerPedido(req, res) {
  const id = req.query.id;                          // 1 fuente
  const filtro = `id = '${id}'`;                    // 2 propagación
  const sql = `SELECT * FROM pedidos WHERE ${filtro}`;
  const filas = await db.query(sql);                // 3 sumidero
  res.json(filas);
}

// --- corrección: consulta parametrizada ---------------
export async function obtenerPedido(req, res) {
  const id = Number(req.query.id);                  // 4 saneamiento
  if (!Number.isInteger(id)) return res.status(400).end();
  const filas = await db.query(
    "SELECT * FROM pedidos WHERE id = $1", [id]  // 5 sin camino
  );
  res.json(filas);
}
```

1 — El dato entra sin tipo ni límite

Cualquier texto de la URL llega a la variable.

2 — El dato cambia de nombre

El analizador de flujo lo sigue; el de patrón pierde el rastro aquí.

3 — Se concatena en una consulta

Con id = 1' OR '1'='1 la consulta devuelve toda la tabla.

4 — La conversión corta el camino

Un entero no lleva comillas ni operadores.

5 — El parámetro va separado de la consulta

El motor recibe la consulta y el valor por canales distintos.

Falsos positivos, falsos negativos y qué hacer con cada uno

Situación	Causa habitual	Acción
Aviso sin riesgo real	El saneamiento es una función propia que el analizador no conoce	Declarar la función como saneadora en la configuración de reglas
Aviso sin riesgo real	Código de prueba o de ejemplo analizado junto al productivo	Excluir rutas de prueba; mantener el alcance en src
Aviso sin riesgo real	Regla genérica que no aplica al framework usado	Desactivar la regla con razón escrita y versionada
Riesgo sin aviso	Framework o librería sin modelo de fuentes y sumideros	Agregar reglas propias para las llamadas del framework
Riesgo sin aviso	Defecto de lógica: autorización, flujo de negocio	Pruebas y revisión de código; el analizador no lo cubre

1

Un hallazgo silenciado sin razón escrita vale lo mismo que uno no detectado

La exclusión vive en el repositorio con los cuatro datos de la sesión 4.

2

El indicador del analizador es el porcentaje de avisos que terminaron en una acción

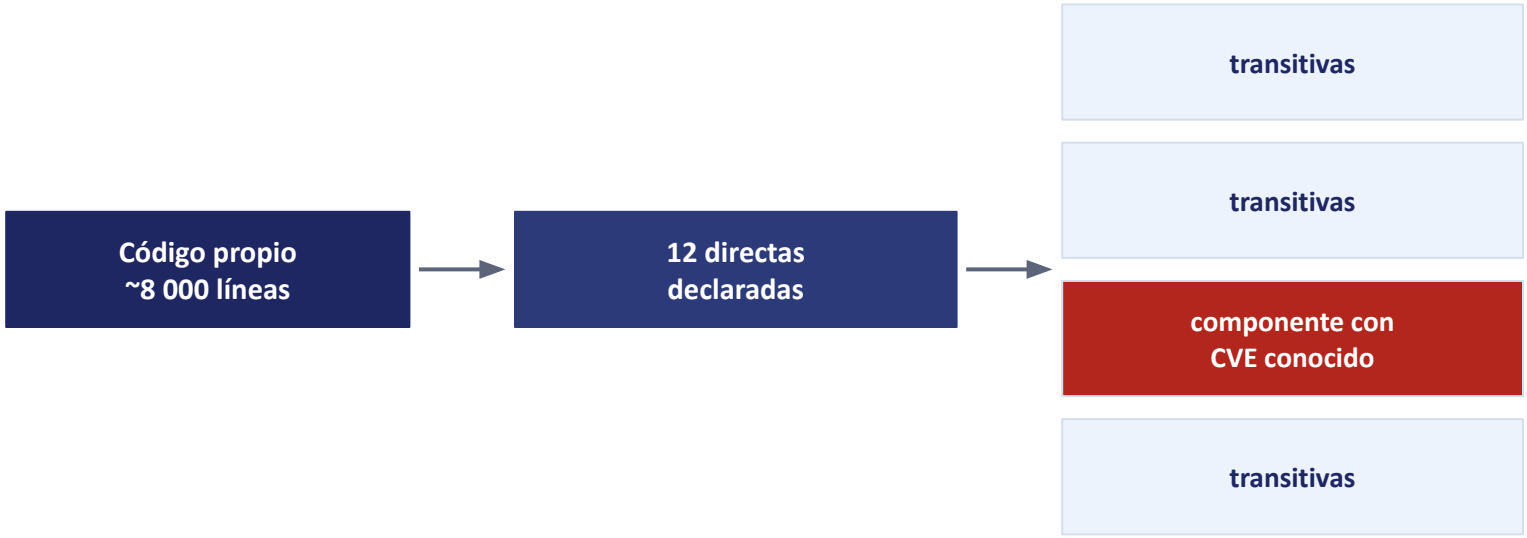
Por debajo del 30 % el equipo deja de leer el reporte; la calibración de reglas vuelve a empezar.

B

Librerías y componentes

El código que el equipo no escribió y que representa la mayor parte del artefacto

El árbol de dependencias y de quién es cada rama



Cifras ilustrativas de un servicio web mediano en Node

~380 componentes en el lock file

Qué hace el análisis

- Lee el lock file: el árbol resuelto y fijado, con las versiones exactas.
- Compara cada componente y versión contra bases de vulnerabilidades conocidas.
- Reporta el camino: qué dependencia directa trajo el componente afectado.

1 **La corrección sube la dependencia directa que trae el componente**

Editar el lock file a mano se pierde en la siguiente instalación. Si la directa no tiene versión corregida, se declara una sustitución con razón y vencimiento.

2 **Sin lock file el análisis responde sobre un árbol que puede cambiar mañana**

Es el mismo requisito de reproducibilidad del Laboratorio 2, ahora como condición del control.

El inventario del artefacto: SBOM

Qué es

Lista de todos los componentes que viajan dentro del artefacto, con nombre, versión, proveedor y relación entre ellos. Se genera en el pipeline a partir del lock file y de la imagen, en un formato estándar legible por máquina.

Para qué sirve

Cuando se publica una vulnerabilidad nueva, la pregunta es qué artefactos en producción la contienen. Con el inventario guardado junto a cada digest la respuesta sale de una consulta en minutos.

1. **Se genera después del build y se guarda con el digest de la imagen**
   Inventario y artefacto quedan unidos: el inventario de la versión 2.3 describe exactamente la imagen 2.3.

2. **Sin inventario, cada vulnerabilidad nueva exige volver a escanear todo lo desplegado**
   Con decenas de servicios eso toma días. En 2021 una librería de registro de Java lo demostró: con inventario, la respuesta tomó horas.

3. **Es un entregable del pipeline y un requisito contractual creciente**
   Clientes y reguladores empiezan a pedirlo junto con el software; la fuente SSDF lo ubica en la práctica PW.4.

Severidad y prioridad son dos números distintos

Pregunta	Qué mira	Efecto sobre la prioridad
Severidad	Puntaje publicado con la vulnerabilidad, calculado sin conocer el sistema	Punto de partida; por sí sola ordena mal
¿Es alcanzable?	Si el código propio llama a la función vulnerable del componente	Una crítica no alcanzable baja; una media alcanzable sube
¿Está expuesta?	Si el componente procesa entrada de internet o corre en una red interna	La misma falla pesa distinto en un servicio público y en un job nocturno
¿Se está explotando?	Catálogos públicos de vulnerabilidades con explotación activa confirmada	Si figura ahí, se corrige en la semana aunque la severidad sea media
¿Hay corrección?	Versión parchada disponible o solo mitigación	Sin parche, la acción es mitigar y registrar; con parche, subir la versión

Un hallazgo crítico en un componente que el código nunca invoca, en un job interno, sin explotación conocida, se registra y se planifica. Uno medio, alcanzable, en la API pública y con explotación activa, detiene el pipeline.

Componentes maliciosos: lo que ninguna base de CVE lista

Vector	Cómo ocurre	Control en el pipeline
Nombre parecido	Un paquete con una letra cambiada respecto del popular, publicado en el registro público	Lock file con hash de integridad; registro privado que replica solo lo aprobado
Confusión de dependencia	Un paquete interno sin publicar y otro público con el mismo nombre y versión mayor	Alcance de nombres reservado y registro interno con prioridad
Mantenedor comprometido	Una versión nueva de un paquete legítimo incluye código malicioso	Versiones fijadas; actualizaciones por revisión, no automáticas
Script de instalación	El paquete ejecuta código al instalarse, antes de que nada lo use	Instalación sin scripts por defecto en el runner
Acción de pipeline alterada	La acción referenciada por tag cambia de contenido; el workflow la ejecuta con los secretos	Referenciar acciones por digest de commit, igual que una imagen

El pipeline ejecuta con credenciales código que el equipo no escribió: paquetes y acciones. Fijar la versión exacta y verificar su integridad es el mismo principio del digest aplicado a lo que entra al runner.

C

Análisis de contenedores

Qué viaja dentro de la imagen además del código, y en qué punto del pipeline se revisa

Qué hay dentro de una imagen y quién revisa cada capa

Código y configuración propios	→	SAST, detección de secretos, revisión
Dependencias de la aplicación	→	Análisis de composición sobre el lock file
Runtime del lenguaje	→	Análisis de imagen: versión del runtime y sus CVE
Paquetes del sistema operativo	→	Análisis de imagen: base de datos de paquetes de la distribución
Imagen base	→	Elección de base, política de actualización y reconstrucción

arriba: lo que el equipo escribe · abajo: lo que la imagen base trae

El análisis de imagen cubre las tres capas inferiores. Sin él, el pipeline revisa el 2 % del artefacto que escribió el equipo y deja sin mirar el resto.

Menos superficie: la decisión está en la imagen base

Imagen base	Paquetes del SO	Consecuencia
Distribución completa	**300 a 500**	Shell, compiladores y utilidades que la aplicación no usa; decenas de hallazgos cada mes
Variante reducida (slim)	**80 a 120**	Lo necesario para ejecutar; sigue habiendo shell y gestor de paquetes
Sin distribución (distroless)	**10 a 20**	Solo runtime y librerías mínimas; sin shell: depurar exige otra imagen
Base del proveedor serverless	**según proveedor**	Trae el cliente de runtime obligatorio; la elección se reduce a la versión

1

La imagen se reconstruye aunque el código no cambie

La base recibe correcciones cada semana; una imagen de hace dos meses tiene los hallazgos de hace dos meses. Un build programado la pone al día.

2

La base se fija por digest y se actualiza por revisión

Igual que una dependencia: versión exacta, cambio visible en el repositorio, análisis antes de adoptarla.

Tres momentos para revisar la imagen



1 · Antes de publicar

- Analiza la imagen recién construida contra el umbral del gate.
- Si falla, no hay imagen en el registro: nada que desplegar.
- Genera el inventario y lo guarda junto al digest.

2 · En el registro, de forma continua

- Una imagen aprobada ayer recibe hallazgos nuevos hoy.
- El registro vuelve a evaluar lo publicado y avisa qué imágenes desplegadas están afectadas.
- Dispara la reconstrucción programada.

3 · Al desplegar

- El entorno verifica la firma antes de aceptar la imagen.
- Solo despliega lo que produjo el pipeline y pasó el análisis.
- Una imagen construida a mano en un equipo queda fuera.

La firma une los tres momentos: certifica que este digest salió de este pipeline después de pasar estas compuertas. Sin firma, el registro acepta cualquier imagen con el nombre correcto.

La secuencia completa de controles sobre el artefacto

GITHUB ACTIONS — CONTROLES DE SEGURIDAD EN CI

```
jobs:
  secretos:      { ... }                        # detiene
  estatico:                                     # 1
    steps:
      - uses: <analizador-estatico>@<sha>
        with: { reglas: alta-confianza, alcance: src/ }
  dependencias:                                 # 2
    steps:
      - uses: <analizador-composicion>@<sha>
        with: { severidad-minima: alta, solo-con-parche: true }
      - run: generar-sbom --salida sbom.json
  imagen:
    needs: [secretos, estatico, dependencias]   # 3
    steps:
      - run: docker build -t app:${{ github.sha }} .
      - uses: <analizador-imagen>@<sha>          # 4
        with: { severidad-minima: alta, ignorar-sin-parche: true }
      - run: firmar app:${{ github.sha }}        # 5
      - run: docker push ... && guardar-sbom --digest $DIGEST
```

1. **Acciones fijadas por digest de commit**
   El workflow ejecuta solo el contenido revisado.

2. **El umbral declara qué detiene**
   Severidad mínima y existencia de parche, versionados.

3. **La imagen se construye solo si CI aprobó**
   Una falla arriba no deja imagen que analizar.

4. **El análisis corre antes del push**
   Nada con hallazgo sobre el umbral llega al registro.

5. **Firma e inventario viajan con el digest**
   Lo que se despliega lleva su prueba y su lista de partes.

ACTIVIDAD

Actividad · Triage de hallazgos

Un pipeline corrió y dejó estos seis hallazgos. Para cada uno decidan: detiene, informa o excepción; la acción concreta; y quién la autoriza.

#	Control	Hallazgo	Datos
1	Estático	Concatenación de parámetro HTTP en consulta SQL, en el módulo de consulta de saldos	Flujo completo; sin saneamiento; API pública
2	Estático	Uso de función de hash retirada para nombres de archivos temporales	Patrón; sin dato de entrada externo
3	Composición	Crítica en componente de compresión, transitiva de la librería de PDF	No alcanzable: la función vulnerable no se usa; parche disponible
4	Composición	Media en el framework HTTP, en el manejo de cabeceras	Alcanzable; expuesta; en catálogo de explotación activa; parche disponible
5	Imagen	Catorce hallazgos altos en paquetes del sistema operativo de la base	Base con tres meses sin reconstruir; parches disponibles en la base actual
6	Cadena	Tres acciones del workflow referenciadas por tag mayor	Una de ellas con acceso a los secretos de despliegue

Decisión	Acción	Quién autoriza	Dato que pesó
detiene · informa · excepción	corregir · subir versión · reconstruir · fijar	la política · el equipo · el equipo con quien responde por el riesgo	alcance · exposición · explotación · parche

Entregable verbal: un grupo por hallazgo explica su decisión en un minuto.

Puntos clave

1. **El analizador estático sigue el dato de la fuente al sumidero**
   Lo que el equipo declara como saneamiento define qué tan útil es el reporte.

2. **La mayor parte del artefacto la escribió otro**
   El análisis de composición lee el lock file y reporta el camino hasta la dependencia directa.

3. **La prioridad combina alcanzabilidad, exposición, explotación y parche**
   La severidad publicada es el punto de partida; las otras cuatro preguntas ordenan la cola.

4. **La imagen se analiza antes de publicar, en el registro y al desplegar**
   La firma une los tres momentos al digest; el inventario responde qué contiene.

5. **Lo que entra al runner con credenciales se fija por digest**
   Paquetes, acciones e imagen base: la misma regla para los tres.